6주차

클로드 코드에 API 날개 달기

API 설정, AI 모델 연동, '냉장고를 부탁해' 앱 제작

신구대학교 컴퓨터소프트웨어학과 · 앱프로그래밍응용 (2026-1)

오늘 배울 내용

이 수업의 3대 핵심 역량

역량	설명
바이브 코딩	AI 코딩 파트너를 활용, 자연어 프롬프트로 빠르게 개발
컴퓨팅 사고	문제 분해 → 패턴 인식 → 추상화 → 알고리즘적 해결
프로젝트 관리	요구사항 → 일정 계획 → 중간 보고 → 최종 발표

오늘의 학습 목표

1. **API**의 개념을 이해하고 AI 모델 API를 선택할 수 있다.
2. **오픈라우터**에서 API 키를 발급받고 클로드 코드에 안전하게 등록할 수 있다.
3. **이미지 인식과 텍스트 생성 API**를 연동한 실제 웹 애플리케이션을 만들 수 있다.

> 📖 *교재: 혼자 공부하는 바이브 코딩 with 클로드 코드 — Chapter 06 (p.222~255)*

지난 주 복습

5주차에서 배운 것

- **할루시네이션** — AI가 그럴듯하지만 사실과 다른 정보를 생성하는 현상, 교차 검증으로 대응
- **커스텀 명령어** — `.claude/commands/`에 마크다운 파일로 반복 작업 자동화
- **인수와 플레이스홀더** — `$ARGUMENTS`, `$1`, `$2`로 명령어에 유연성 부여
- **전역 vs 프로젝트 명령어** — 홈 디렉터리(전역)와 작업 폴더(프로젝트)에 따른 적용 범위 차이
- **명령어 체이닝** — `&&` 연산자로 여러 커스텀 명령어를 순서대로 연결 실행 (교재 p.210)
- **통합 커스텀 명령어** — `/quiz-daily`처럼 여러 단계를 하나의 파이프라인으로 묶어 일일 업데이트 자동화 (교재 p.213)
- **유지보수 워크플로** — 기존 커스텀 명령어만 선택적으로 수정하여 전체 시스템을 효율적으로 관리 (교재 p.217)

지난 주 복습 (계속)

이번 주 핵심 질문

> 💬 *"지금까지 만든 프로그램은 모두 미리 준비된 데이터로 작동했는데, **외부 AI 모델의 힘을 빌리려면** 어떻게 해야 할까?"*

5주차까지 커스텀 명령어로 작업을 자동화하는 방법을 배웠습니다. 이번 주부터는 **외부 AI 모델을 API로 연동**하여 이미지 인식, 텍스트 생성 같은 강력한 기능을 내 앱에 추가합니다!

이번 주의 컴퓨팅 사고

컴퓨팅 사고의 **4가지 사고력**이 이번 주에 어떻게 적용되는지 살펴봅시다.

사고력	의미	이번 주 적용
분해	큰 문제를 작은 단위로 나누기	'냉장고를 부탁해' 앱을 3단계 PRD로 분해
패턴 인식	반복되는 구조 찾기	API 호출 패턴(키 설정→모델 선택→요청→응답) 인식
추상화	핵심만 뽑아 정리하기	.env 파일로 API 키를 추상화, 코드와 설정 분리
알고리즘	순서대로 절차 밟기	이미지 업로드→재료 인식→레시피 생성→저장 순서

💡 이번 주의 핵심은 **"분해"**! 복잡한 다중 AI 모델 연동 앱을 3단계 PRD로 쪼개서 점진적으로 완성하는 것이 컴퓨팅 사고의 실전 적용입니다.

3대 역량, 이번 주 적용

역량	이번 주 활동
바이브 코딩	클로드 코드에 API 연동 프롬프트를 입력하여 '냉장고를 부탁해' 앱 제작
컴퓨팅 사고	API 호출 구조를 분해하고, .env로 보안 설정을 추상화
프로젝트 관리	3단계 PRD로 개발 계획을 수립하고 단계별로 점진적 구현

바이브 코딩 = 프롬프트로 API 연동 앱 **구현** (도구)
컴퓨팅 사고 = API 구조를 **분해·추상화**하여 이해 (사고)
프로젝트 관리 = 3단계 PRD로 **계획 수립·점진적 개발** (계획)

API의 개념과 활용 방법 (교재 p.225)

API란?

API(Application Programming Interface)는 프로그램과 AI 모델을 이어주는 **다리**입니다.

- 마치 전화로 전문가에게 도움을 요청하듯, 프로그램이 AI 모델에게 사진을 분석해 달라고 요청하면 AI가 이미지를 해석하고 결과를 돌려줍니다.
- 웹 애플리케이션은 사용자가 **업로드할 인터페이스만 제공**하고, 핵심 AI 기능은 API가 처리

API 활용 예시: '냉장고를 부탁해'

```
사용자 → [냉장고 사진 업로드] → 앱 → [API] → AI 모델(이미지 인식)
                                                  ↓
사용자 ← [레시피 추천 표시] ← 앱 ← [API] ← AI 모델(텍스트 생성)
```

💡 복잡한 AI 모델을 직접 만들 필요 없이, API를 통해 이미 구축된 고성능 AI 모델을 바로 사용할 수 있습니다.

AI 모델 선택하기 (교재 p.226)

AI 모델을 사용하는 두 가지 방법

방법	설명	예시
API 통합 플랫폼	여러 AI 모델을 한곳에서 비교·선택	오픈라우터(OpenRouter)
각 회사 공식 API	특정 회사 API를 직접 연동	OpenAI, Anthropic, Google

오픈라우터(OpenRouter)란?

- 여러 AI 모델 API를 **한곳에 모아** 제공하는 API 통합 플랫폼
- 클로드, GPT, 라마(Llama) 등 다양한 AI 모델을 **하나의 API 키**로 사용 가능
- **무료 모델**도 많아 학습 목적으로 충분히 활용 가능

> 초보자라면 **오픈라우터**로 시작하는 것을 추천합니다. 여러 모델을 쉽게 비교하고, 무료 모델로 체험할 수 있는 데다 한 곳에서 모든 모델을 관리할 수 있기 때문입니다. (교재 p.227)

API 키란? (교재 p.227)

API 키 = 비밀 열쇠

API 키(API key)는 API를 사용할 때 사용자를 인증하는 **비밀 열쇠**입니다.

- API를 **안전하고 공정하게** 사용하기 위한 인증과 관리의 핵심 도구
- 서버는 API 키를 통해 **누가, 얼마나** 요청을 보냈는지 확인

API 키가 없다면?

- 누구나 서버에 접근 → 데이터 유출 위험 🚨
- 사용량 추적 불가 → 과도한 요청 제한 불가 🚨

API 키 보안 규칙

- API 키를 **코드에 직접 하드코딩하지 말 것** ❌
- **.env 파일**에 저장하고, **.gitignore**에 등록하여 Git에 올리지 않을 것 ✅
- 공유 폴더가 아닌 **개인 로컬 컴퓨터**에만 보관할 것 ✅

오픈라우터에서 API 키 발급받기 (교재 p.227~231)

발급 절차

1. **openrouter.ai** 접속 → [Sign in] 클릭
2. **구글 계정**으로 가입 (또는 이메일 가입)
3. 구글 아이디 입력 → Continue → 약관 동의
4. 우측 상단 **프로필 아이콘** → [Keys] 메뉴 선택
5. **[Create API Key]** 버튼 클릭
6. Name에 `HonGong-Vibe` 입력 → [Create] 클릭
7. 생성된 키를 **복사하여 안전한 곳에 저장** (한 번만 표시!)

.env 파일에 API 키 저장

```
OPENROUTER_API_KEY=sk-or-v1-여기에발급받은키붙여넣기
```

- 메모장에서 위 내용을 입력 → [다른 이름으로 저장]
- 파일 이름: `.env` / 파일 형식: **모든 파일(*.*)** / 인코딩: UTF-8

무료 AI 모델 탐색하기 (교재 p.232~235)

'냉장고를 부탁해' 앱에 필요한 모델 2가지

기능	필요한 모델	선택한 모델
이미지 인식	사진 속 재료를 분석	Google: Gemma 3 27B (free)
텍스트 생성	재료로 레시피 추천	Qwen: Qwen 3.6 Plus (free)

오픈라우터에서 무료 모델 찾기

1. 상단 메뉴에서 **[Models]** 클릭
2. 왼쪽 사이드바 **'Prompt pricing'** 필터 → 슬라이더를 **FREE**에 맞추기
3. **Input Modalities** 필터에서 [Image] 선택 → 이미지 인식 모델 탐색
4. [Text] 선택 → 텍스트 생성 모델 탐색
5. 각 모델 페이지에서 **[📋복사]** 아이콘으로 모델명 복사 → 텍스트 파일에 저장

> 💡 모델 검색 결과는 수시로 변동됩니다. 교재에서 제시한 모델이 보이지 않으면 원하는 모델을 선택해도 됩니다. (교재 p.235)

무료 AI 모델 탐색하기 — 최신 변동 사항

⚠️ **2026년 4월 기준 변동 사항**: 교재의 DeepSeek V3.1 무료(:free) 버전은 **종료**되었습니다. 텍스트 생성용으로 **Qwen 3.6 Plus(:free)**를 사용하세요. 이미지 인식용 Gemma 3 27B(:free)는 정상 운영 중이며, 후속 모델 **Gemma 4**(26B/31B)도 출시되었습니다.

무료 텍스트 모델 대안 (2026.04 기준)

모델	컨텍스트	특징
Qwen 3.6 Plus (free) ⭐	1M	1순위 추천, 멀티모달
OpenAI GPT-OSS 120B (free)	131K	OpenAI 오픈소스
MiniMax M2.5 (free)	204K	코딩/에이전트 특화
Meta Llama 3.3 70B (free)	131K	다국어 지원

AI 모델 연결하고 테스트하기 (교재 p.236~241)

프로젝트 준비

1. `VibeCoding` 폴더에 `Study-04` 폴더 생성
2. 터미널에서 `claude` 실행 → `/init` 입력
3. CLAUDE.md 파일 자동 생성 확인

API 키를 클로드 코드에 등록

```
OpenRouter에서 받은 API 키를 .env 파일로 저장했어.
이 키를 안전하게 사용할 수 있도록 준비해 줘.
```

AI 모델 연결하고 테스트하기 (계속)

클로드 코드가 생성하는 파일들

파일	역할
`.gitignore`	.env 파일이 Git에 커밋되지 않도록 보호
`.env.example`	다른 개발자들을 위한 환경 변수 템플릿
`config.js`	환경 변수를 안전하게 로드하고 검증하는 유틸리티
`example.js`	OpenRouter API를 사용하는 예제 코드
`package.json`	Node.js 프로젝트 설정 파일

.env와 .gitignore 파일의 이해 (교재 p.239)

.env 파일

- 프로그램의 **설정값을 저장**하는 특별한 파일
- API 키처럼 민감한 정보를 프로그램 코드와 **분리하여 관리**
- 코드 수정 없이 `.env` 파일만 바꾸면 설정 변경 가능

.gitignore 파일

- `.env` 파일이 **깃허브에 업로드되지 않도록** 보호하는 설정 파일
- 클로드 코드가 API 키 설정 시 **자동으로** `.gitignore`에 `.env`를 등록

API 연동 테스트 프롬프트 (교재 p.239)

```
이제 준비된 API가 실제로 작동하는지 테스트해 줘.
이미지 인식은 google/gemma-3-27b-it:free 모델을 이용하고,
텍스트 생성은 qwen/qwen3.6-plus:free 모델을
이용할 거야.
API를 통한 텍스트와 이미지 인식을 각각 테스트해서
실행 결과를 알려줘.
```

💡 오픈라우터에서 API를 적용할 때는 반드시 **모델명을 정확히 입력**해야 합니다. (교재 p.239)

✏️ 셀프 체크 — 06-1 클로드 코드에서 API 설정하기

(교재 p.242)

문제 1. 오픈라우터에서 API를 사용하기 위해 필요한 것을 **모두** 고르세요.

① API 키　　② 모델명
③ 자체 서버　　④ .gitignore 파일

문제 2. 클로드 코드에서 API 키를 안전하게 관리하기 위한 방법이 **아닌** 것을 고르세요.

① .env 파일에 API 키 저장
② .gitignore 파일에 .env 추가
③ 코드에 직접 API 키 하드코딩
④ 프로젝트 폴더 내에서만 키 참조

✏️ 셀프 체크 — 06-1 (계속)

(교재 p.242)

문제 3. 오픈라우터의 Activity 메뉴에서 확인할 수 있는 정보가 **아닌** 것은?

① 사용한 모델명
② 사용 시간
③ 호출한 코드 내용
④ API 호출 기록

'냉장고를 부탁해' 앱 소개 (교재 p.243)

FridgeChef — AI 기반 맞춤형 레시피 추천 시스템

냉장고 사진을 업로드하면 **AI가 재료를 인식**하고, **맞춤 레시피를 추천**해 주는 앱입니다.

앱의 핵심 기능

기능	사용 AI 모델	설명
🔍 **재료 인식**	Gemma 3 27B-it (이미지 인식)	냉장고 사진에서 식재료를 자동 분석
🍳 **레시피 생성**	Qwen 3.6 Plus (텍스트 생성)	인식된 재료로 맞춤 레시피 추천
💾 **레시피 저장**	—	사용자 프로필에 레시피 저장·관리

다중 AI 모델 연동

- 이처럼 **두 가지 이상의 AI 모델**이 필요한 복잡한 앱을 만들 때는
- 전체 프로젝트를 **작은 단위로 나누어** 점진적으로 개발하는 것이 효율적

단계별 계획 수립하기 (교재 p.244)

3단계 PRD 분해 전략

```
PRD_step1.md → 이미지 인식 핵심 기능 (5일)
PRD_step2.md → 레시피 생성 시스템 (7일)
PRD_step3.md → 사용자 프로필 및 저장 기능 (10일)
```

PRD	핵심 기능	AI 모델
Step 1	Streamlit 기반 웹 UI + 냉장고 재료 인식	Gemma 3 27B-it
Step 2	레시피 생성 + 재료 편집 + 필터링 및 검색	Qwen 3.6 Plus
Step 3	사용자 인증(회원가입/로그인) + 레시피 저장 + 대시보드	—

> **컴퓨팅 사고 연결**: 전체 앱을 한 번에 만들지 않고 **3단계로 분해**하여 점진적으로 구현합니다. 각 단계는 이전 단계를 기반으로 발전하며, 최종적으로 완성도 높은 웹 애플리케이션이 됩니다.

클로드 코드에서 PRD 생성하기 (교재 p.244~246)

PRD 생성 프롬프트

```
앞서 만든 OpenRouter API를 이용해서, 냉장고 사진에서
재료를 인식하고 레시피를 추천하는 웹 애플리케이션을
만들고 싶어. 다음과 같이 3단계로 나누어서 PRD를 만들어 줘.

1단계는 이미지를 입력받고 google/gemma-3-27b-it:free
모델을 사용해서 이미지를 인식해.
2단계는 1단계에서 얻은 정보와 qwen/qwen3.6-plus:free
모델을 사용해서 레시피를 생성해.
3단계는 사용자 프로필을 만들어서 레시피를 저장해.

각 단계를 PRD_step1.md, PRD_step2.md, PRD_step3.md로
나누어 저장해 줘.
```

생성 결과

- `Study-04` 폴더에 3개의 PRD 문서가 생성됨
- 각 단계는 이전 단계를 기반으로 점진적으로 발전

1단계: 냉장고 이미지에서 재료 인식하기 (교재 p.246~249)

프롬프트

```
PRD_step1.md를 실행해.
```

구현 결과

- **Streamlit** 웹 프레임워크로 웹 인터페이스 구축
- 이미지 업로드 → Gemma 모델이 자동으로 재료 분석
- 재료명, 수량, 상태가 **카드 형태**로 표시

테스트 프롬프트

```
메인 애플리케이션을 실행해서 1단계의 결과를 테스트해 줘.
```

💡 **웹 프레임워크**(Streamlit)란 웹 애플리케이션을 쉽게 만들 수 있도록 도와주는 도구 모음입니다. 파이썬 코드만으로 웹 앱을 만들 수 있어 AI 모델 시연에 매우 유용합니다. (교재 p.247)
📌 최신 버전: `pip install streamlit` (v1.56.0, 2026.04 기준, Python ≥ 3.10 필요)

2단계: 레시피 생성하기 (교재 p.250~251)

프롬프트

```
이제 PRD_step2.md를 실행해 줘.
```

구현 결과

- Qwen 3.6 Plus 모델을 연동하여 **레시피 생성** 기능 추가
- 재료 편집 기능과 레시피 표시 인터페이스 구현
- 사이드바에 [레시피 생성] 메뉴 추가

테스트 프롬프트

```
메인 애플리케이션을 실행해서 2단계 결과를 테스트하게 해 줘.
```

레시피에 포함되는 정보

- 요리명, 난이도, 조리시간, 칼로리
- 필요한 재료, 조리 순서, 예상 시간

3단계: 사용자 프로필에 레시피 저장하기 (교재 p.252~254)

프롬프트

```
이제 PRD_step3.md를 실행해 줘.
```

구현 결과

- 로그인/회원가입 시스템 (프로토타입)
- 개인 대시보드: 저장한 레시피, 요리 횟수, 사용 통계
- [저장] 버튼으로 마음에 드는 레시피 보관
- [나의 레시피] 탭에서 저장된 레시피 확인

로컬 개발 환경의 이해 (교재 p.254)

- 현재 앱은 **로컬 환경**(내 컴퓨터)에서만 작동 (`localhost`)
- 다른 사람은 접속할 수 없음 → 실제 서비스를 위해서는 **배포** 과정이 필요
- 지금은 **로컬에서의 개발과 테스트**에 집중!

✏️ 셀프 체크 — 06-2 '냉장고를 부탁해'

(교재 p.255)

문제 1. PRD를 3단계로 분리하여 개발할 때, 2단계에서 텍스트 모델(Qwen 등) API 호출이 실패하면 다음 중 점검이 필요한 항목을 **모두** 고르세요.

① .env 파일의 OPENROUTER_API_KEY 존재 여부
② 텍스트 모델명의 정확성
③ 1단계에서 Streamlit 서버가 이미 8501 포트를 사용 중인지 여부
④ .gitignore 파일의 존재 여부

문제 2. Streamlit과 전통적인 웹 개발(HTML/CSS/JS)의 차이점으로 **틀린** 것을 고르세요.

① Streamlit은 파이썬 코드만으로 웹 앱을 만들 수 있다.
② Streamlit은 데이터 시각화와 ML 모델 시연에 특화되어 있다.
③ Streamlit은 반드시 프런트엔드와 백엔드를 분리해야 한다.
④ Streamlit은 `streamlit run` 명령어로 실행 가능하다.

✏️ 셀프 체크 — 06-2 (계속)

(교재 p.255)

문제 3. 프로젝트에서 API 키가 깃허브에 업로드되는 것을 방지하는 구체적인 방법을 서술하세요.

> ✅ .env 파일에 API 키를 저장하고, .gitignore 파일에 .env를 등록하여 Git 버전 관리에서 제외합니다. 클로드 코드가 API 키 설정 시 이 과정을 자동으로 처리해 줍니다.

실습을 시작합니다!

지금부터 **외부 AI 모델을 API로 연동**하고, **'냉장고를 부탁해' 앱**을 만들어 봅니다.

실습 흐름

```
① 오픈라우터 가입 & API 키 발급 → ② .env 파일 생성 →
③ 무료 AI 모델 선택 → ④ API 연동 테스트 →
⑤ PRD 생성 → ⑥ 1단계 재료 인식 → ⑦ 2단계 레시피 생성 →
⑧ 3단계 저장 기능
```

준비물

- 인터넷 연결된 PC (Windows)
- 명령 프롬프트 (cmd) 또는 터미널
- 구글 계정 (오픈라우터 가입용)
- `VibeCoding/Study-04` 폴더 생성

실습 ① 오픈라우터 가입 & API 키 발급

1단계: 오픈라우터 회원가입

1. **openrouter.ai** 접속 → [Sign in] 클릭
2. 구글 로고 클릭 → 구글 아이디 입력 → Continue

2단계: API 키 발급

1. 우측 상단 프로필 아이콘 → [Keys] 메뉴
2. [Create API Key] 클릭
3. Name에 `HonGong-Vibe` 입력 → [Create]
4. 생성된 API 키를 **반드시 복사하여 안전하게 보관!**

3단계: .env 파일 생성

1. 메모장에서 `OPENROUTER_API_KEY=발급받은키` 입력
2. [다른 이름으로 저장] → `Study-04` 폴더 이동
3. 파일 이름: `.env` / 파일 형식: **모든 파일** / 인코딩: UTF-8

실습 ② 무료 AI 모델 탐색 & 선택

이미지 인식 모델 찾기

1. 오픈라우터 상단 [Models] 클릭
2. 'Prompt pricing' → FREE로 설정
3. 'Input Modalities' → [Image] 선택
4. **Google: Gemma 3 27B (free)** 클릭 → 모델명 복사하여 텍스트 파일에 저장

텍스트 생성 모델 찾기

1. 'Input Modalities' → [Text] 선택
2. **Qwen: Qwen 3.6 Plus (free)** 클릭 → 모델명 복사하여 텍스트 파일에 저장

> 💡 Qwen 3.6 Plus가 안 보이면 검색창에 `qwen 3.6 plus`로 검색하세요. `(:free)`를 포함하면 검색이 안 됩니다!

실습 ② 무료 AI 모델 탐색 (확인)

[확인] 체크리스트

- [ ] 오픈라우터에 정상 가입되었는가?
- [ ] API 키가 발급되고 안전하게 저장되었는가?
- [ ] 이미지 인식 모델명과 텍스트 생성 모델명이 텍스트 파일에 저장되었는가?

실습 ③ API 연동 테스트

클로드 코드 실행 & 초기 설정

```
cd VibeCoding/Study-04
claude
```

```
/init
```

API 키 등록 프롬프트

```
OpenRouter에서 받은 API 키를 .env 파일로 저장했어.
이 키를 안전하게 사용할 수 있도록 준비해 줘.
```

실습 ③ API 연동 테스트 (프롬프트)

API 연동 테스트 프롬프트

```
이제 준비된 API가 실제로 작동하는지 테스트해 줘.
이미지 인식은 google/gemma-3-27b-it:free 모델을 이용하고,
텍스트 생성은 qwen/qwen3.6-plus:free 모델을
이용할 거야.
API를 통한 텍스트와 이미지 인식을 각각 테스트해서
실행 결과를 알려줘.
```

실습 ③ API 연동 테스트 (확인)

[확인] 체크리스트

- [ ] `.gitignore`, `.env.example`, `config.js`, `example.js`, `package.json` 파일이 생성되었는가?
- [ ] 텍스트 인식 모델이 정상 작동하는가? (한국어 텍스트 분석 요청 응답 확인)
- [ ] 이미지 인식 모델이 정상 작동하는가? (테스트 이미지 인식 결과 확인)

오픈라우터에서 사용 기록 확인

1. 오픈라우터 홈페이지 → 프로필 아이콘 → [Activity]
2. Timestamp 열에 **Gemma와 Qwen 모델** 사용 기록 확인

> 💡 API 키를 프롬프트에 직접 입력해 작업하는 것은 보안상 문제로 권하지 않습니다. 반드시 **.env 파일**을 통해 관리하세요. (교재 p.237)

실습 ④ PRD 생성 & 1단계 구현

PRD 생성 프롬프트

```
앞서 만든 OpenRouter API를 이용해서, 냉장고 사진에서
재료를 인식하고 레시피를 추천하는 웹 애플리케이션을
만들고 싶어. 다음과 같이 3단계로 나누어서 PRD를 만들어 줘.
```

1단계 실행 프롬프트

```
PRD_step1.md를 실행해.
```

1단계 테스트

```
메인 애플리케이션을 실행해서 1단계의 결과를 테스트해 줘.
```

[확인] 체크리스트

- [ ] PRD_step1.md, PRD_step2.md, PRD_step3.md가 생성되었는가?
- [ ] 웹 브라우저에서 이미지 업로드 → 재료 인식이 작동하는가?

실습 ⑤ 2단계 레시피 생성 & 3단계 저장

2단계 실행

```
이제 PRD_step2.md를 실행해 줘.
```

```
메인 애플리케이션을 실행해서 2단계 결과를 테스트하게 해 줘.
```

3단계 실행

```
이제 PRD_step3.md를 실행해 줘.
```

[확인] 체크리스트

- [ ] [레시피 생성] 메뉴에서 재료 기반 레시피가 생성되는가?
- [ ] 로그인/회원가입이 작동하는가?
- [ ] 레시피 [저장] 버튼이 동작하고 [나의 레시피]에서 확인되는가?
- [ ] 대시보드에서 사용 통계가 표시되는가?

실습 중 자주 발생하는 문제

문제	원인	해결 방법
API 키 인식 안 됨	.env 파일 경로 오류 또는 형식 오류	`Study-04` 폴더에 `.env` 파일 존재 확인, 파일 형식이 "모든 파일"인지 확인
모델 호출 실패	모델명 오타	오픈라우터에서 모델명을 다시 복사 (대소문자, 콜론 주의)
Streamlit 서버 안 뜸	포트 충돌 또는 패키지 미설치	다른 터미널에서 이미 실행 중인지 확인, `pip install streamlit` 실행
이미지 인식 결과 영어만 출력	1단계 프로토타입 한계	정상 동작, 2단계에서 한국어 레시피 생성 가능
.env 파일이 Git에 올라감	.gitignore 미설정	`.gitignore`에 `.env` 추가 확인
텍스트 모델 호출 실패	모델 종료 또는 일시 장애	1순위: `qwen/qwen3.6-plus:free`, 2순위: `openai/gpt-oss-120b:free`로 교체

핵심 키워드 — 이론 (교재 p.242)

> 📖 *5가지 키워드로 끝내는 핵심 포인트*

- **API**(Application Programming Interface) — 외부 AI 서비스와 프로그램을 연결하는 다리로, 간단한 요청만으로 복잡한 AI 기능을 활용할 수 있게 해 주는 통신 규칙
- **API 통합 플랫폼** — 여러 AI 모델을 한 곳에서 비교하고 선택할 수 있는 서비스 (예: 오픈라우터)
- **API 키**(API key) — API 서비스에 접근하기 위해 발급받아 사용자를 인증하는 비밀 열쇠
- **.env** — API 키가 저장된 환경 변수 파일로, API를 호출할 때마다 참조됨
- **.gitignore** — .env 파일이 자동으로 깃허브에 업로드되지 않도록 특정 파일을 제외하여 보호하는 설정 파일

핵심 키워드 — 실습 (교재 p.255)

> 📖 *2가지 키워드로 끝내는 핵심 포인트*

- **다중 AI 모델 연동** — 이미지 인식, 텍스트 생성 등 목적에 맞게 여러 AI 모델을 활용하는 방식으로, 애플리케이션을 제작할 때는 단계별 계획에 따라 점진적으로 완성도를 높여야 함
- **웹 프레임워크** — 웹 애플리케이션을 쉽게 만들 수 있도록 도와주는 도구 모음으로, Streamlit은 파이썬 코드만으로 AI 모델 시연용 웹 앱을 빠르게 구축 가능

오늘의 마무리 체크리스트

수업이 끝나기 전에 아래 항목을 확인해 보세요!

- [ ] **API의 개념**을 이해하고 "프로그램과 AI 모델의 다리"라고 설명할 수 있다
- [ ] **오픈라우터** 가입 및 **API 키 발급** 완료
- [ ] **.env 파일**에 API 키를 저장하고 **.gitignore**로 보호한 이유를 설명 가능
- [ ] **무료 AI 모델 2가지**(이미지 인식 + 텍스트 생성)를 탐색하고 선택함
- [ ] **API 연동 테스트**에 성공하고 오픈라우터 Activity에서 사용 기록 확인
- [ ] **3단계 PRD**를 생성하고 단계별로 '냉장고를 부탁해' 앱 구현
- [ ] "내가 오늘 배운 것" 2줄 메모

> ✅ FridgeChef 앱 실행 화면 캡처 + API 연동 테스트 결과 캡처를 교수님께 제출하면 오늘 수업 완료!

최신 업데이트 — API 연동의 진화 (2026년 4월 기준)

> 📢 *교재 출간 이후 주요 업데이트를 정리합니다!*

오픈라우터 & AI 모델 변동

구분	교재 (2025)	최신 (2026.04)
이미지 인식	Gemma 3 27B (free) ✅	정상 운영, 후속 **Gemma 4** (26B/31B) 출시
텍스트 생성	DeepSeek V3.1 (free) ❌	무료 버전 종료 → **Qwen 3.6 Plus (free)** 사용
DeepSeek 최신	V3.1	**V3.2** / V3.2 Speciale 출시 (2025.12)
오픈라우터 SDK	없음	`@openrouter/sdk` 베타 출시 (Node.js)

Claude Code 새 기능

- `/model` — 대화 중 사용 모델 변경 가능 (`/model` 명령어)
- `/config` — 설정 관리 인터페이스 (권한, 환경 변수, 도구 설정)
- **MCP 서버** — 외부 도구(DB, 파일 시스템, API)를 구조화해 연동

최신 업데이트 (계속)

Claude Code 새 기능 (계속)

- **플러그인 / 서브에이전트** — 기능 확장 및 역할별 AI 에이전트 구성
- **데스크톱 앱 & 웹 버전** — 터미널 외에 데스크톱, 웹, VS Code에서 사용

> 💡 기존 .env 방식은 그대로 호환됩니다. 수업에서는 교재대로 .env 파일을 사용하세요!

과제 안내

이번 주 과제

'냉장고를 부탁해' 앱의 3단계까지 구현을 완료하고, 직접 냉장고 사진으로 테스트하세요.

과제 요구사항

- 자신의 **냉장고 사진**을 촬영하여 앱에 업로드
- AI가 인식한 **재료 목록** 캡처
- 생성된 **레시피 결과** 캡처
- **오픈라우터 Activity** 화면 캡처 (API 호출 기록)
- 실행 결과 **스크린샷** 제출

다음 주 예고

> ***7주차**: AI 서브에이전트로 개발팀 만들기 — 코드 리뷰, 최적화, UX 디자인 역할 분담*

오늘 배운 "API 연동"과 "다중 AI 모델 활용"은 앞으로 **실제 서비스형 앱을 만드는 데 필수적인 기술**입니다. 이제 외부 AI의 힘을 빌려 더 강력한 애플리케이션을 만들 수 있습니다!